

Image Captioning with Vision-Language Models

Pedagogical Report

INFO 7390: Advanced Data Science and Architecture

Northeastern University | December 2024

Prepared by: Abhinav Kumar Piyush

1. Introduction

Image captioning is the task of automatically generating natural language descriptions of images. It combines computer vision (understanding images) and NLP (generating text) into a single multimodal system. This workshop teaches the complete pipeline from theory to production.

Why Image Captioning Matters

Application	Impact	Scale
Accessibility	Alt-text for visually impaired	285M people worldwide
Social Media	Auto-captioning posts	2B+ images daily
E-commerce	Product descriptions	80% manual effort reduction
Healthcare	Radiology report assistance	Faster diagnosis
Search Engines	Image indexing	Billions of images searchable

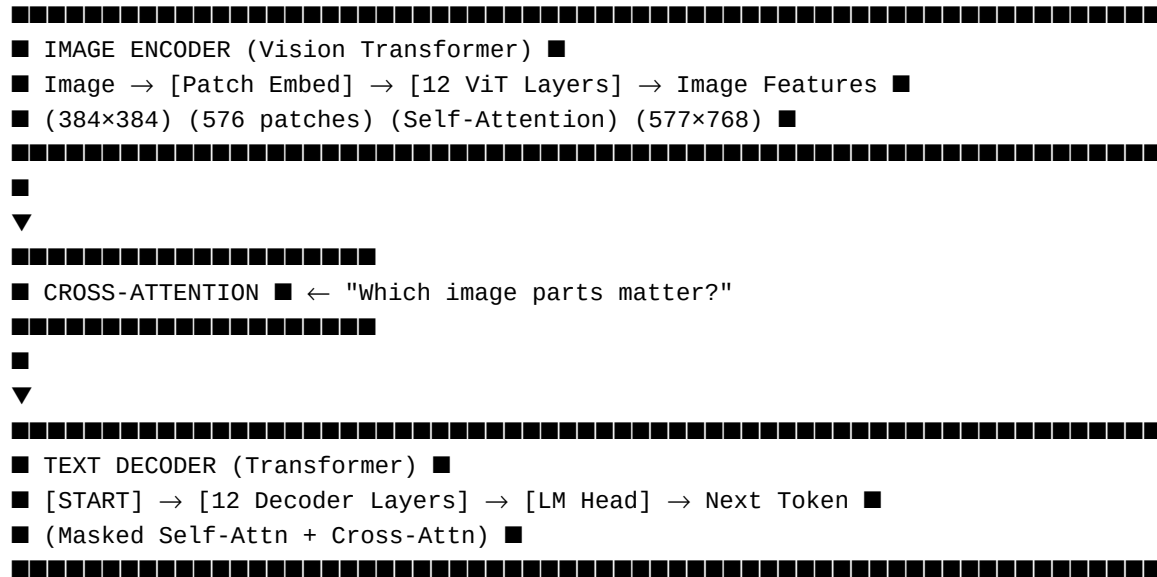
Workshop Objectives

#	Learning Objective	Bloom's Level
1	Explain encoder-decoder architecture and cross-attention	Understanding
2	Implement image captioning using pre-trained BLIP	Applying
3	Analyze generation parameter effects on output quality	Analyzing
4	Evaluate captions using BLEU and semantic similarity	Evaluating
5	Apply GIGO principles for failure mode detection	Applying
6	Design production-ready pipelines with error handling	Creating

2. Technical Architecture

Image captioning uses an encoder-decoder architecture. The encoder (Vision Transformer) processes the image into feature vectors. The decoder (Transformer) generates text word-by-word, using cross-attention to focus on relevant image regions.

ENCODER-DECODER ARCHITECTURE



Vision Transformer (ViT)

ViT treats images as sequences of patches. A 384×384 image is split into 576 patches (24×24 grid of 16×16 patches). Each patch is embedded into a 768-dim vector. The sequence passes through 12 transformer layers with self-attention, allowing patches to exchange information.

Cross-Attention Mechanism

Cross-attention bridges vision and language. The Query comes from the text decoder ('what am I looking for?'), while Key and Value come from the image encoder ('what does each patch contain?'). The attention weights determine which image regions influence each generated word.

Formula: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) \times V$

3. BLIP Model

BLIP (Bootstrapping Language-Image Pre-training) from Salesforce is our model of choice. It offers excellent zero-shot performance, manageable size (~385M parameters), and easy integration with Hugging Face.

Component	Specification
Vision Encoder	ViT-B/16, 12 layers, 768 hidden dim
Text Decoder	12 layers, 30,522 vocabulary
Total Parameters	~385 Million
Training Data	129M image-text pairs
Input Resolution	384×384 or 224×224
Inference Speed	~100ms (GPU) / ~5s (CPU)

Generation Parameters

Text generation can use beam search (deterministic, consistent) or sampling (stochastic, creative). Key parameters control the output:

Parameter	Range	Effect
num_beams	1-10	Higher = better quality, slower
temperature	0.1-2.0	Higher = more creative/random
top_p	0.5-1.0	Nucleus sampling threshold
max_length	10-100	Maximum caption length
repetition_penalty	1.0-2.0	Prevents word repetition

4. Implementation

Project Structure

```
image-captioning-workshop/  
├── notebooks/  
│   ├── Image_Captioning_Tutorial.ipynb # Main tutorial  
│   └── exercises/  
│       ├── starter_template.ipynb # Practice  
│       └── solutions.ipynb # Answers  
├── app/  
│   └── gradio_demo.py # Interactive demo  
├── src/  
│   └── captioner.py # Reusable module  
├── README.md  
└── requirements.txt
```

Core Code Example

```
from transformers import BlipProcessor, BlipForConditionalGeneration  
  
# Load model (downloads ~1GB on first run)  
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")  
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")  
  
# Generate caption  
inputs = processor(image, return_tensors="pt")  
output = model.generate(**inputs, max_length=30, num_beams=5)  
caption = processor.decode(output[0], skip_special_tokens=True)
```

Technology Stack

Technology	Purpose
PyTorch	Deep learning framework
Hugging Face Transformers	Model loading and inference
Gradio	Interactive web demo
NLTK	BLEU score calculation
Sentence-Transformers	Semantic similarity

5. GIGO Principles

Garbage In, Garbage Out is critical for multimodal AI. Unlike text where typos are obvious, image quality issues are subtle. A blurry, dark, or corrupted image will produce poor captions without obvious errors.

Quality Checks

Check	Threshold	Why It Matters
Resolution	Min 100×100 px	Low-res images lose detail
Aspect Ratio	Max 3:1	Extreme ratios indicate cropping issues
Brightness	30-240 (0-255)	Too dark/bright loses information
Color Variance	Min std 15	Low variance = blank/corrupted

Confidence Estimation

Even with good inputs, models can be uncertain. We estimate confidence by generating multiple captions with sampling and measuring consistency. High consistency = confident model. Divergent samples = uncertain, flag for review.

```
# Generate 5 samples
samples = [generate_caption(image, do_sample=True, temperature=0.8) for _ in range(5)]

# Measure pairwise similarity
consistency = average_pairwise_similarity(samples)
# consistency > 0.8 = confident, < 0.5 = uncertain
```

6. Evaluation Metrics

No single metric captures caption quality. We teach two complementary approaches:

BLEU Score

BLEU measures n-gram overlap between generated and reference captions. BLEU-1 checks single words, BLEU-4 checks 4-word phrases. Fast and interpretable, but misses synonyms ('dog' vs 'canine' = 0 overlap).

Semantic Similarity

Uses sentence embeddings to capture meaning regardless of exact words. 'A dog on the beach' and 'A canine by the ocean' score high despite few shared words. More meaningful but slower.

Metric	Pros	Cons
BLEU	Fast, interpretable, standard	Misses synonyms, paraphrases
Semantic Similarity	Captures meaning, handles paraphrases	Slower, requires embedding model

When Metrics Disagree

Test Case	BLEU	Semantic	Interpretation
Identical sentences	1.0	1.0	Both agree
Paraphrases	0.2	0.9	Semantic wins - same meaning
Word order change	0.8	0.3	BLEU fooled - different meaning
Completely different	0.0	0.1	Both agree

7. Teaching Approach

The Explain-Show-Try Framework

Explain: Theoretical foundations with diagrams and analogies. We explain 'why' not just 'what'.

Show: Live code with detailed comments. Copy-paste ready but educational.

Try: Progressive exercises from fill-in-the-blank to open-ended implementation.

Exercise Progression

Exercise	Pts	Skill	Task
1: Basic Caption	10	API usage	Write function returning caption
2: Conditional	15	Prompts	Test different prompt styles
3: Benchmarking	20	Analysis	Measure parameter effects
4: Quality Analyzer	25	Implementation	Build GIGO checker
5: Full Pipeline	30	Design	Production-ready class
Bonus: Confidence	+10	Advanced	Multi-sample consistency

Course Theme Connections

INFO 7390 Theme	Application in Workshop
GIGO	Quality analyzer validates inputs before captioning
Botspeak	Conditional prompts guide model output style
Computational Skepticism	Multi-metric evaluation, confidence estimation

8. Limitations & Future Work

Current Limitations

- **Domain Specificity:** BLIP trained on natural images; performance degrades on medical/satellite imagery
- **English Only:** No multilingual support without additional models
- **Hallucination Risk:** Model can generate plausible but incorrect details
- **Compute Requirements:** CPU inference is 10-20x slower than GPU

Future Improvements

- **Attention Visualization:** Show which regions influenced each word
- **Fine-tuning Tutorial:** Domain adaptation for specific use cases
- **VQA Extension:** Visual Question Answering tasks
- **Multi-model Comparison:** Compare BLIP, GIT, LLaVA architectures

9. Conclusion

This workshop provides a comprehensive introduction to image captioning, balancing theoretical depth with practical skills. Students learn not just how to generate captions, but how to evaluate them critically, detect problematic inputs, and build production-ready systems.

The explicit connections to course themes — GIGO, Botspeak, Computational Skepticism — reinforce that technical skills must be paired with critical thinking. The architectural patterns covered (encoder-decoder, cross-attention, GIGO validation) transfer directly to other multimodal tasks.

10. References

- [1] Li, J., et al. (2022). BLIP: Bootstrapping Language-Image Pre-training. ICML 2022. arXiv:2201.12086
- [2] Dosovitskiy, A., et al. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021. arXiv:2010.11929
- [3] Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762
- [4] Papineni, K., et al. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL 2002.
- [5] Reimers, N., & Gurevych, I. (2019). Sentence-BERT. EMNLP 2019. arXiv:1908.10084
- [6] Lin, T.Y., et al. (2014). Microsoft COCO: Common Objects in Context. ECCV 2014.
- [7] Hugging Face Transformers Documentation. <https://huggingface.co/docs/transformers>
- [8] BLIP Model Card. <https://huggingface.co/Salesforce/blip-image-captioning-base>

Workshop Materials

- GitHub Repository: [Your GitHub URL]
- Interactive Demo: Run `python gradio_demo.py`
- Video Tutorial: [Your Video URL]

